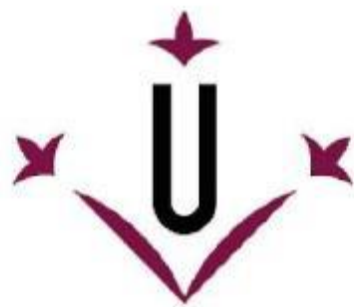




Universitat de Lleida

PROJECTE CATALAN NUMBERS

ESTRUCTURA DE DADES – PRA2

Abraham Ruiz Mosteirín – Míriam Rodríguez Carranza

Índex

Implementació Context.....	3
Implementació Mètode Catalan Iter.....	3
Implementació ArrayStack	4
public class ArrayStack<E> implements Stack<E> {	4
public ArrayStack() {	4
Mètode push {.....	4
Mètode top {.....	5
Mètode pop {.....	5
Mètode isEmpty	5
Mètode resize {	6
Conclusions.....	6
Eficiència i Flexibilitat.....	6
Complexitat espacial i temporal.....	6
Limitacions.....	6

Implementació Context

La classe Context es defineix com una classe estàtica interna que emmagatzema informació de cada context de trucada (com paràmetres i variables locals), útil en una implementació iterativa que emula una recursiva. Els atributs de n : el valor del número Catalan, el res que és l'acumulador per emmagatzemar el resultat parcial de Catalan i Catalan1 com a variable auxiliar en el càlcul iteratiu, el qual, és l'índex de la posició al càlcul, i guarda un valor intermedi de Catalan. L'entryPoint el qual es capaç de determinar en quin punt del càlcul es troba el context, permet múltiples etapes d'execució al càlcul.

La classe Context representa una delimitació en el càlcul de números de Catalan i permet que cada etapa mantingui el seu propi estat.

Implementació Mètode CatalanIter

Aquest mètode calcula el número de Catalan corresponent a n de manera iterativa, utilitzant una fórmula basada en combinatòria que depèn de valors anteriors de la seqüència. Implementació: Inicialitza un arranjament català[] de mida " $n+1$ ", que servirà com a memòria intermèdia per a l'emmagatzemament de cada número de Catalan fins a $C(n)$. Això permet evitar el re càlcul dels valors ja calculats, optimitzant el temps d'execució.

Estableix el cas base $C(0) = 1$, ja que, el número de Català per a 0 és 1. El primer bucle controla el càlcul per a cada valor, des de 1 fins a n . El segon bucle, calcula el valor de Catalan [i]. Finalment, el mètode torna Catalan, contenint el número de Catalan per al valor n sol·licitat.

El mètode CatalanIter, implementa un enfocament de programació per al càlcul de números de Catalan, evitant les múltiples trucades recursives mitjançant l'emmagatzematge de resultats parcials en una solució.

Implementació ArrayStack

La implementació de “*ArrayStack*”, implementa la interfície “*Stack<E>*”, cosa que significa que segueix el comportament estàndard d'una pila: LIFO (*Last In, First Out*).

public class ArrayStack<E> implements Stack<E> {

```
private E[] elements; //Array per emmagatzemar els elements de la pila
public int size; //Grandària actual de la pila
private static final int DEFAULT_CAPACITY = 10; // Capacitat inicial per defecte
```

}

És un array que emmagatzema els elements de la pila.

public ArrayStack() {

```
elements = (E[]) new Object [DEFAULT_CAPACITY]; // Inicialitza l'array
size = 0; // Grandària inicial de la pila
```

}

El constructor crea un array amb una capacitat predeterminada “*DEFAULT_CAPACITY*” de 10. Això assegura que s'utilitzi una memòria mínima fins que l'array necessiti redimensionar-se.

Mètode push {

```
@Override
public void push(E elem) {
    if (size == elements.length) {
        resize(2 * elements.length); // Duplica la mida de l'array si és ple
    }
    elements[size++] = elem; // Afegeix l'element i augmenta la mida
```

}

El mètode “*push*”, afegeix un element a la part superior de la pila. El seu redimensionament el fa quan l'array està ple i invoca al mètode “*resize*” per a duplicar la mida de l'array, permetent el creixement de la pila. Això garanteix que la pila pot emmagatzemar més elements dels inicialment mostrats sense cap problema.

Mètode top {

```
@Override public E top() {  
  
    if (isEmpty()) { throw new NoSuchElementException("Stack is empty");  
  
    }  
  
    return elements[size - 1]; // Retorna l'element a la part superior  
  
}
```

El mètode “*top*”, retorna l'element a la part superior de la pila sense eliminar-lo, la seva verificació de buidatge, fa que en la cas de que la pila sigui buida, llençar una excepció “*NoSuchElementException*” per dirigir aquest error. Això fa que el retorn de l'element serà l'últim element inserit usant elements “[*size - 1*]” per apuntar a la posició superior de la pila.

Mètode pop {

```
@Override public void pop() {  
  
    if (isEmpty()) { throw new NoSuchElementException("Stack is empty");  
  
    }  
  
    elements[--size] = null; // Elimina el darrer element i redueix la mida  
  
}
```

El mètode “*pop*”, elimina l'element a la part superior de la pila. Després, fa una verificació de buidatge ja que, en cas de que la pila estigui buida, llança una excepció “*NoSuchElementException*”. I a l'hora de l'eliminació de l'element, decrementarà el “*size*” i també, eliminant la referència a l'últim element assignant-lo cap a *null*. Això ajuda a alliberar la memòria de l'element suprimit.

Mètode isEmpty

```
@Override public boolean isEmpty() {  
  
    return size == 0; // Retorna true si la pila és buida  
  
}
```

El mètode “*isEmpty*” ens indica si la pila és buida. *True* en cas de “*size == 0*”, indicant que no conté elements.

Mètode `resize` {

```
@SuppressWarnings("unchecked")
```

```
private void resize(int capacity) {
```

```
    E[] newArray = (E[]) new Object[capacity]; // Crea un nou array amb la nova capacitat
```

```
    System.arraycopy(elements, 0, newArray, 0, size); // Copia els elements al nou array
```

```
    elements = newArray; // Reemplaça l'array antic
```

```
}
```

El mètode “*resize*” ajusta la mida de l'array quan es requereix més espai. Aquest mètode s'invoca només quan l'array està ple. Normalment, es crea un nou array “*newArray*” amb la capacitat especificada segons la “*CAPACITY*”, i els elements de l'array original es copien a aquest nou array. Finalment, al reemplaçament de l'anterior, apuntaran al nou array generat.

Conclusions

Eficiència i Flexibilitat

La classe “*ArrayStack*” permet un creixement de la pila mitjançant el redimensionament de l'array quan es plena. Això la converteix en una estructura de pila flexible, adequada per a situacions on s'espera un nombre variable d'elements.

Complexitat espacial i temporal

L'operació “*push*” té una complexitat mitjana d' $O(n)$ en temps, excepte quan es requereix redimensionar l'array, cost d' $O(n)$ degut a la còpia d'elements. L'operació “*pop*” i “*top*” són d' $O(1)$.

Limitacions

Les operacions de redimensionament poden ser costoses en termes de temps i memòria. A més a més, quan l'array s'ha de duplicar cada vegada que s'assoleix la seva capacitat, pot haver-hi fragmentació de memòria si es fan molts “*push*” i “*pop*” amb un gran nombre d'elements.

Implementació tests ArrayStack

Excepció en fer pop en una pila buida

Codi: *pop_on_empty_stack_should_throw_nse_exception* .

Verifica que trucar a *pop()* en una pila buida llanci una excepció *NoSuchElementException*.

Comprovació d'excepció en utilitzar top en una pila buida

Codi: *top_on_empty_stack_should_throw_nse_exception*

Assegura que trucar a *top()* en una pila buida també llanci una *NoSuchElementException*.

Obtenció de l'element superior

Codi: *push_and_top*

Després d'afegir elements, comprova que *top()* retorna l'últim element empès.

Verificació de l'augment

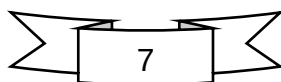
Codi: *push_increases_size*

Assegura que la mida de la pila s'incrementi correctament amb cada operació *push()*.

Remoure l'element superior

Codi: *pop_removes_top_element*

Verifica que *pop()* elimina l'element superior i que *top()* retorna el següent element correcte.



Disminuir mida

Codi: *pop_decreases_size*

Comprova que la mida de la pila disminueix adequadament després d'una operació *pop()*

Verificació de pila buida

Codi: *is_empty_on_new_stack*

Verifica que una instància nova d'*ArrayStack* estigui buida.

Redimensionament de la matriu

Codi: *push_resizes_array*

Assegura que la pila redimensioni la matriu interna quan s'empenyen més elements que la capacitat inicial, i que la mida sigui correcta.